INTEGRAL DRAGON

AI-ACCELERATED SOFTWARE DEVELOPMENT • FOUNDATIONS

Foundations Overview

Modules 1 through 4 — the operating baseline for trustworthy AI-accelerated delivery.

AI-Accelerated Software Development | Foundations Program | v2.0

Confidential. Prepared exclusively for Ensemble Health Partners.

Foundations is not a theory block — it is the operating baseline

- Replace ad hoc AI usage with a disciplined engineering operating model
- Establish the CLI-native working environment before acceleration features are added
- Teach control surfaces — specs, plans, rules, permissions, review — before scope expands
- Prepare learners to enter Labs with repeatable habits, not one-off prompt tricks

The goal is not faster prompts.
The goal is controlled, reviewable, enterprise-ready engineering work.

The shift: from vibe coding to structured agentic engineering

	Vibe Coding (Legacy approach)	Agentic Engineering (This program's target)
Interface	— Conversational prompting via external chatbots. Heavy copy-paste, no codebase awareness.	✓ CLI-native orchestration. Agent reads the codebase, executes shell commands, and loops on failure.
Workflow Trigger	— High-level, unconstrained prompts driving a "creative flow." Ideas prioritized over structure.	✓ Spec-Driven Development: intent → constraints → acceptance criteria. Small, verifiable prompts.
Constraint Mechanism	— Model generates without limits. Arbitrary network calls, unchecked mutations, architectural drift.	✓ Explicit functional boundaries. CLAUDE.md rules, Plan Mode gating, and hook enforcement.
Quality & Review	— Minimal oversight. "Trust debt" accumulates as engineers can't reconstruct why code exists.	✓ Structured review discipline. AI output treated as untrusted until verified by tests or human review.
Security Posture	— Brittle, opaque codebases. High risk of OWASP vulnerabilities, no audit trail for agent actions.	✓ Zero Trust execution. Least-privilege permissions, immutable audit logs, OWASP-aware threat model.

Enterprise environments cannot tolerate vibe coding at scale. This curriculum instills the discipline that makes AI-accelerated delivery safe to ship.

The four-module arc: each layer adds control before adding power

MODULE 1 · WEEK 1
The Agentic Paradigm and CLI Environment

Sessions 1–4 · ~92 min

- **S01** The Agentic Engineering Shift
- **S02** LLM Internals for Orchestrators
- **S03** Context Boundaries and Rot
- **S04** CLI Architecture and Setup

Adopt the agentic posture and understand the tool. Mental model before execution.

MODULE 2 · WEEK 1
Spec-Driven Development and Context Control

Sessions 5–8 · ~92 min

- **S05** Spec-Driven Development
- **S06** Plan Mode vs. Act Mode
- **S07** Context Control via CLAUDE.md
- **S08** Extending Capabilities with MCP

Constrain intent, execution, and context. Explicit intent before generation.

MODULE 3 · WEEK 2
Tooling, MCP, and Agentic Orchestration

Sessions 9–12 · ~92 min

- **S09** Skills and Deterministic Hooks
- **S10** Task Decomposition and Subagents
- **S11** Parallel Orchestration: Agent Teams
- **S12** OWASP Top 10 for Agentic Apps

Extend capability and delegate work safely inside established constraints.

MODULE 4 · WEEK 2
Enterprise Security, Governance, and Auditability

Sessions 13–16 · ~92 min

- **S13** Zero Trust and Permission Boundaries
- **S14** Secure MCP Governance
- **S15** Code Review and Cognitive Debt
- **S16** Auditability and Workflow Narration

Secure, govern, and produce auditable evidence. Enterprise use requires all four layers.

→ Each module assumes the previous one is in place. Control before capability is the sequence.

Two weeks · four modules · sixteen sessions · ~23 minutes each

Week 1

MENTAL MODEL · WORKFLOW

Module 1 — The Agentic Paradigm and CLI Environment

S01–S04

S01 · Agentic Engineering Shift | S02 · LLM Internals | S03 · Context Boundaries & Rot | S04 · CLI Architecture & Setup

Dismantle the vibe coding mindset. Establish the CLI harness and shared vocabulary for agentic orchestration.

Module 2 — Spec-Driven Development and Context Control

S05–S08

S05 · Spec-Driven Development | S06 · Plan Mode vs. Act Mode | S07 · Context via CLAUDE.md | S08 · MCP Capabilities

Never let an agent write code without a spec. Separate planning from mutation. Persist project rules across all sessions.

Week 1 exit state: Learners operate in the CLI with bounded context, written specs, and reviewed plans.

Week 2

TOOLING · SECURITY & GOVERNANCE

Module 3 — Tooling, MCP, and Agentic Orchestration

S09–S12

S09 · Skills & Hooks | S10 · Task Decomposition | S11 · Parallel Agent Teams | S12 · OWASP Top 10

Extend the harness with deterministic enforcement, delegated subagents, and parallel coordination. Secure the expanded surface.

Module 4 — Enterprise Security, Governance, and Auditability

S13–S16

S13 · Zero Trust Permissions | S14 · Secure MCP Governance | S15 · Code Review & Debt | S16 · Auditability & Narration

Apply least privilege, govern tool boundaries, review AI diffs for structural risk, and build evidence trails that survive audit.

Week 2 exit state: Learners can orchestrate, secure, and narrate agentic work. Ready for Labs.

The Agentic Paradigm and CLI Environment

Session 1: The Agentic Engineering Shift

Replace casual AI usage with a structured model: output, reviewability, and ownership all matter simultaneously. Comprehension debt is introduced as the first enterprise risk.

Session 2: LLM Internals for Orchestrators

Build mechanical sympathy — tokens, inference, variance, context windows — to make better workflow decisions. Only the theory that directly impacts engineering choices.

Session 3: Context Boundaries and Rot

Understand why context quality matters more than context volume. The minimum working set principle, the N-tool problem, and how overloaded agents lose focus and coherence.

Session 4: Claude Code CLI Architecture and Setup

Install and correctly bound the harness — context init, tool injection, trust boundary, and approval mode. The CLI becomes the primary engineering environment, not a supplementary tool.

Core message: AI writes syntax; engineers still own architecture, judgment, and liability.

Spec-Driven Development and Context Control

Session 5: Spec-Driven Development

Define machine-readable intent — goal, constraints, acceptance criteria, non-goals — before generation begins. Code without a preceding spec is a policy violation, not a shortcut.

Session 6: Plan Mode vs. Act Mode

Separate codebase analysis from mutation. Generate a plan in read-only mode, reject it, refine it, then approve before any file is touched. Prevents rogue agents from rewriting architecture.

Session 7: Context Control via CLAUDE.md

Persist project rules so constraints survive across sessions, engineers, and agents without re-prompting. Standardizes AI behavior across teams. Scoped: user, project, local, and managed.

Session 8: Extending Capabilities with MCP

Connect external systems through governed tool boundaries — client/server architecture, payload inspection, authorization, and blast-radius control. MCP is the 2026 enterprise standard.

Core message: Reliable agentic engineering starts with explicit intent and governed context.

Tooling, MCP, and Agentic Orchestration

Session 9: Skills and Deterministic Hooks

Decide what remains flexible (skills that scale team expertise) versus what becomes automatic enforcement (hooks that fire deterministically before every commit or file write).

Session 10: Task Decomposition and Subagents

Break large work into bounded delegated units. Give each subagent a clear contract, scope limit, and output interface. Prevent context collapse through atomic task design.

Session 11: Parallel Orchestration: Agent Teams

Coordinate multiple agents simultaneously without losing state discipline, collision control, or merge coherence. The N×M orchestration problem becomes manageable with explicit boundaries.

Session 12: OWASP Top 10 for Agentic Apps

Secure the broader execution surface created by tooling, teams, and expanded capability. Prompt injection, indirect tool calls, unauthorized escalation, and MCP server compromise.

Core message: Once the workflow is constrained, capability can expand safely.

Enterprise Security, Governance, and Auditability

Session 13: Zero Trust and Permission Boundaries

Apply least privilege to agent execution — trust no implicit scope, verify all access. Configure role-based boundaries so subagents cannot escalate beyond their defined mission.

Session 14: Secure MCP Governance

Govern external tool and MCP trust boundaries — authorization tokens, payload inspection, blast radius containment, and server compromise detection.

Session 15: Code Review and Cognitive Debt Mitigation

Review AI-generated diffs for structural risk and comprehension debt, not just surface correctness. A passing test is not a substitute for a reviewable change.

Session 16: Auditability and Workflow Narration

Preserve enough evidence to reconstruct why a change happened — narration discipline, commit traceability, and the compliance record that enterprise delivery requires.

Core message: Enterprise use is not real until permissions, review, and traceability hold under scrutiny.

The control stack — from posture to auditable evidence

1

Posture & Mental Model

Output · Reviewability · Ownership · Mechanical sympathy · Vibe coding vs. engineering

2

Bounded Context & Explicit Intent

Specs · Plan Mode · CLAUDE.md rules · Minimum working set · Governed MCP

3

Deterministic Enforcement & Decomposition

Hooks · Subagents · Agent teams · OWASP threat model · Atomic scoping

4

Permissions, Review & Auditable Evidence

Zero trust · MCP governance · Cognitive debt review · Narration & traceability

- Each layer depends on the previous. Skipping the posture layer makes the permission layer theater.
- This is the path from AI assistance to enterprise-grade agentic engineering.

Session progression at a glance

S01–S04 Week 1 · M1	Posture, mechanics, context quality, and CLI harness setup. The mental model and the environment — before any tooling is layered on.
S05–S08 Week 1 · M2	Specs, plans, persistent project rules, and governed external tools. Intent becomes explicit; context becomes bounded; project standards get versioned.
S09–S12 Week 2 · M3	Hooks, subagents, parallel agent teams, and the OWASP threat surface. Capability expands deliberately inside the constraints already in place.
S13–S16 Week 2 · M4	Zero trust permissions, MCP governance, cognitive debt review, and auditable narration. Enterprise-grade delivery requires all four layers simultaneously.

→ **Each module adds control before adding more power.** The sequence is intentional and non-negotiable.

What learners can do by the end of Foundations

- ✓ Explain the difference between vibe coding and structured agentic engineering — and why it matters for enterprise delivery
- ✓ Drive work from specs and reviewed plans rather than open-ended prompts
- ✓ Decompose work across tools and agents with clear delegation contracts and scope limits
- ✓ Choose the smallest useful context for a task and defend that choice to a reviewer
- ✓ Operate Claude Code inside explicit permission and context boundaries configured by the team
- ✓ Review and narrate AI-assisted changes in terms a team can inherit, audit, and trace back to intent

> Labs assume these capabilities are already in place. Learners who enter Labs without this foundation will be working at a disadvantage from the first exercise.

Labs assume Foundations is already in practice — not just understood

- Labs assume the learner can already work in the CLI without guidance on setup or basics
- Labs assume bounded scope and review discipline are the learner's default posture
- Labs assume AI output is treated as untrusted until verified by tests or review
- Labs build on this foundation with codebase reconnaissance, test generation, and CI loops

> The difference between a learner who struggled through Foundations and one who internalized it will be visible in the first lab exercise.

KEY TAKEAWAY

Control before capability. Structure before speed.

Foundations is the operating baseline for trustworthy AI-accelerated delivery. Every session that follows builds on what this two-week program establishes.